Action Memory in Multi-Agent Reinforcement Learning: Preventing Bang-Bang Control Through State Augmentation

Implementation Report
CaNS Opposition Control with Deep Reinforcement Learning

February 4, 2026

Abstract

This report presents the implementation of action memory in multi-agent deep reinforcement learning for turbulent flow control. We address the bang-bang control problem—characterized by high-frequency oscillations in control actions—through state augmentation with previous action information. We provide a rigorous mathematical formulation, explain the mechanism by which this prevents bang-bang behavior, and address the apparent redundancy in critic inputs. The implementation maintains full backwards compatibility while enabling smooth, adaptive temporal control.

Contents

1 Introduction

1.1 Problem Statement

In multi-agent reinforcement learning for flow control, we observed undesirable bang-bang behavior: the policy exhibits high-frequency oscillations, rapidly switching between extreme action values (e.g., from $+1$ to -1). This behavior manifests as:

1. High temporal gradient: $\|\nabla_t a\|_{\text{RMS}} \approx 0.4$
2. Excessive high-frequency content in FFT spectrum
3. Suboptimal flow control due to actuator limitations and flow inertia

1.2 Root Cause Analysis

Paradox: Bang-bang control achieves high drag reduction, yet is undesirable. Why?

The bang-bang behavior creates a **closed-loop feedback instability**:

1. **Wave excitation**: Bang-bang actuation triggers high-frequency wave dynamics in the turbulent flow
2. **Observation coupling**: These waves appear in velocity measurements $(\mathbf{u}, \mathbf{w})$
3. **Reactive policy**: Policy $\pi(\mathbf{u}, \mathbf{w})$ responds to oscillating observations with oscillating actions
4. **Feedback loop**: Actions reinforce wave dynamics, creating self-sustaining oscillations
5. **Memory-less amplification**: Without action history, policy cannot detect or break this cycle

The problem is NOT poor performance—drag reduction may be excellent. The issues are:

- Actuators cannot physically achieve high-frequency switching
- May exploit numerical artifacts specific to the simulation
- Poor robustness to model mismatch or real-world conditions
- Unsustainable feedback dynamics

Mathematically, at time t the policy receives state $\mathbf{s}_t = [\mathbf{u}_t, \mathbf{w}_t]$ and outputs action $\mathbf{a}_t = \pi(\mathbf{s}_t)$. At time $t + 1$, it has no information about $\mathbf{a}_t$, so it cannot detect the oscillation pattern and learn to dampen it.

1.3 Proposed Solution

We augment the state representation to include the previous action:

$$\mathbf{s}_t = [\mathbf{u}_t, \mathbf{w}_t, \mathbf{a}_{t-1}] \tag{1}$$

This simple modification enables the policy to learn temporal smoothness adaptively while maintaining the Markov property.

2 Mathematical Formulation

2.1 Standard MDP Without Action Memory

The original formulation uses a Markov Decision Process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ where:

$$\mathcal{S} = \mathbb{R}^{2\times 8\times 8} \quad \text{(state space: velocity fields)} \tag{2}$$

$$\mathcal{A} = [-1, 1]^{8\times 8} \quad \text{(action space: control field)} \tag{3}$$

$$P(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t) : \mathcal{S} \times \mathcal{A} \to \mathcal{S} \quad \text{(dynamics)} \tag{4}$$

$$R(\mathbf{s}_t, \mathbf{a}_t) : \mathcal{S} \times \mathcal{A} \to \mathbb{R} \quad \text{(reward)} \tag{5}$$

State representation for each agent i at patch (m, n):

$$\mathbf{s}_t^{(i)} = \begin{bmatrix} \mathbf{u}_t^{(i)} \\ \mathbf{w}_t^{(i)} \end{bmatrix} \in \mathbb{R}^{2\times 8\times 8} \tag{6}$$

The policy network learns:

$$\mathbf{a}_t^{(i)} = \pi_\theta(\mathbf{s}_t^{(i)}) = \pi_\theta(\mathbf{u}_t^{(i)}, \mathbf{w}_t^{(i)}) \tag{7}$$

Issue: This is memoryless. The policy at time $t + 1$ has no knowledge of $\mathbf{a}_t^{(i)}$.

2.2 MDP With Action Memory

We augment the state space to include previous action:

$$\mathcal{S}_{\text{aug}} = \mathbb{R}^{3\times 8\times 8} \quad \text{(augmented state space)} \tag{8}$$

Augmented state representation:

$$\mathbf{s}_t^{(i)} = \begin{bmatrix} \mathbf{u}_t^{(i)} \\ \mathbf{w}_t^{(i)} \\ \mathbf{a}_{t-1}^{(i)} \end{bmatrix} \in \mathbb{R}^{3\times 8\times 8} \tag{9}$$

The transition dynamics become:

$$P(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t) = P([\mathbf{u}_{t+1}, \mathbf{w}_{t+1}, \mathbf{a}_t]|[\mathbf{u}_t, \mathbf{w}_t, \mathbf{a}_{t-1}], \mathbf{a}_t) \tag{10}$$

Proposition 1 (Markov Property Preservation)**.** The augmented state representation $\mathbf{s}_t = [\mathbf{u}_t, \mathbf{w}_t, \mathbf{a}_{t-1}]$ preserves the Markov property:

$$P(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t) = P(\mathbf{s}_{t+1}|\mathbf{s}_0, \mathbf{a}_0, \ldots, \mathbf{s}_t, \mathbf{a}_t) \tag{11}$$

Proof. The next state depends only on current flow state $(\mathbf{u}_t, \mathbf{w}_t)$ and current action $\mathbf{a}_t$ through the Navier-Stokes equations. The augmented state includes $\mathbf{a}_{t-1}$, which becomes part of the next state representation. Since:

$$\mathbf{s}_{t+1} = [\mathbf{u}_{t+1}, \mathbf{w}_{t+1}, \mathbf{a}_t] \tag{12}$$

and $\mathbf{a}_t$ is determined by the current policy action, the Markov property holds. □

2.3 Policy and Value Functions

The policy network with action memory:

$$\pi_\theta : \mathbb{R}^{3\times 8\times 8} \to [-1,1]^{8\times 8}, \quad \mathbf{a}_t = \pi_\theta(\mathbf{u}_t, \mathbf{w}_t, \mathbf{a}_{t-1}) \tag{13}$$

The state-action value function (Q-function):

$$Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \middle| \mathbf{s}_t = [\mathbf{u}_t, \mathbf{w}_t, \mathbf{a}_{t-1}], \mathbf{a}_t\right] \tag{14}$$

Bellman equation with action memory:

$$Q(\mathbf{s}_t, \mathbf{a}_t) = r_t + \gamma \mathbb{E}_{\mathbf{s}_{t+1}, \mathbf{a}_{t+1}}\left[Q(\mathbf{s}_{t+1}, \mathbf{a}_{t+1})\right] \tag{15}$$

Expanding:

$$Q([\mathbf{u}_t, \mathbf{w}_t, \mathbf{a}_{t-1}], \mathbf{a}_t) = r_t + \gamma \mathbb{E}\left[Q([\mathbf{u}_{t+1}, \mathbf{w}_{t+1}, \mathbf{a}_t], \mathbf{a}_{t+1})\right] \tag{16}$$

Key observation: The next state $\mathbf{s}_{t+1}$ includes the current action $\mathbf{a}_t$. This creates a temporal coupling.

3 Mechanism: How Action Memory Prevents Bang-Bang

3.1 Temporal Coupling in Value Function

Proposition 2 (Temporal Coupling)**.** With action memory, the current action choice $\mathbf{a}_t$ directly affects the future state representation, creating an incentive for temporal consistency.

Proof. Consider two scenarios at time t with identical flow states but different action histories:

$$\text{Scenario A:} \quad \mathbf{s}_t^A = [\mathbf{u}_t, \mathbf{w}_t, +0.9] \tag{17}$$

$$\text{Scenario B:} \quad \mathbf{s}_t^B = [\mathbf{u}_t, \mathbf{w}_t, -0.7] \tag{18}$$

For a given current action $\mathbf{a}_t$, the next states differ:

$$\mathbf{s}_{t+1}^A = [\mathbf{u}_{t+1}, \mathbf{w}_{t+1}, \mathbf{a}_t] \quad \text{if starting from A} \tag{19}$$

$$\mathbf{s}_{t+1}^B = [\mathbf{u}_{t+1}, \mathbf{w}_{t+1}, \mathbf{a}_t] \quad \text{if starting from B} \tag{20}$$

While the flow evolution $(\mathbf{u}_{t+1}, \mathbf{w}_{t+1})$ is identical, the Q-values differ:

$$Q(\mathbf{s}_t^A, \mathbf{a}_t) \neq Q(\mathbf{s}_t^B, \mathbf{a}_t) \quad \text{in general} \tag{21}$$

The Q-function learns to prefer actions that maintain consistency with $\mathbf{a}_{t-1}$ when beneficial. □

3.2 Learning Smooth Control

The policy learns smoothness through three mechanisms:

3.2.1 Mechanism 1: Breaking Closed-Loop Feedback Instability

Critical observation: Bang-bang control actually achieves *high* drag reduction, not poor performance. The problem is more subtle:

1. Bang-bang actuation triggers high-frequency wave dynamics in the flow
2. These waves appear in velocity observations $(\mathbf{u}, \mathbf{w})$
3. Policy sees oscillating observations $\rightarrow$ responds with bang-bang actions
4. Actions reinforce wave dynamics $\rightarrow$ **closed-loop feedback instability**

Mathematically, without action memory:

$$\text{Bang-bang } \mathbf{a}_t \rightarrow \text{Wave dynamics in flow} \tag{22}$$

$$\text{Waves in } (\mathbf{u}_{t+1}, \mathbf{w}_{t+1}) \rightarrow \pi(\mathbf{u}_{t+1}, \mathbf{w}_{t+1}) = \text{Bang-bang } \mathbf{a}_{t+1} \tag{23}$$

$$\rightarrow \text{Self-reinforcing cycle} \tag{24}$$

The reward signal may actually favor bang-bang:

$$\mathbb{E}[R_{\text{bang-bang}}] \geq \mathbb{E}[R_{\text{smooth}}] \quad \text{(bang-bang works well for drag!)} \tag{25}$$

However, bang-bang is undesirable because:

1. Not physically realizable (actuator bandwidth limitations)
2. May exploit numerical artifacts or simulation resonances
3. Poor robustness and generalization to real systems
4. Creates unstable feedback loop with flow dynamics

With action memory, the policy can detect and break this cycle:

$$\pi(\mathbf{u}_t, \mathbf{w}_t, \mathbf{a}_{t-1}) \text{ learns: "I'm oscillating } \rightarrow \text{ dampen the response"} \tag{26}$$

The policy sees the full feedback loop and can learn to stabilize it, trading some drag reduction for smoother, more robust control.

3.2.2 Mechanism 2: Smoothness Loss Regularization

The actor loss includes spatial and consistency penalties:

$$\mathcal{L}_{\text{actor}} = -\mathbb{E}_{\mathbf{s},\mathbf{a}\sim\pi}[Q(\mathbf{s}, \mathbf{a})] + \lambda_s \mathcal{L}_{\text{spatial}} + \lambda_c \mathcal{L}_{\text{consistency}} \tag{27}$$

where:

$$\mathcal{L}_{\text{spatial}} = \mathbb{E}\left[\|\nabla_x \mathbf{a}\|^2 + \|\nabla_y \mathbf{a}\|^2\right] \tag{28}$$

$$\mathcal{L}_{\text{consistency}} = \mathbb{E}\left[\sum_{\substack{i,j: \\ \text{sim}(\phi_i,\phi_j)>\tau}} \max(0, \|\mathbf{a}_i - \mathbf{a}_j\| - m)\right] \tag{29}$$

With action memory, these losses become more effective because the policy can plan ahead:

$$\pi_\theta(\mathbf{u}_t, \mathbf{w}_t, \mathbf{a}_{t-1}) \text{ learns: "Choose } \mathbf{a}_t \approx \mathbf{a}_{t-1} \text{ to minimize future smoothness loss"} \tag{30}$$

3.2.3 Mechanism 3: Value Function Smoothness

The Q-function learns to be smooth in the action-history dimension:

$$\frac{\partial Q([\mathbf{u}, \mathbf{w}, \mathbf{a}_{\text{prev}}], \mathbf{a}_{\text{curr}})}{\partial \mathbf{a}_{\text{curr}}} \text{ becomes coupled with } \mathbf{a}_{\text{prev}} \tag{31}$$

This creates an implicit preference for:

$$\mathbf{a}_{\text{curr}} \approx \mathbf{a}_{\text{prev}} \quad \text{when flow state is similar} \tag{32}$$

3.3 Closed-Loop Feedback Dynamics

Remark 1 (Bang-Bang as Closed-Loop Instability)**.** The bang-bang behavior can be modeled as a feedback system:

$$\text{Flow dynamics:} \quad \mathbf{u}_{t+1} = f_{\text{NS}}(\mathbf{u}_t, \mathbf{a}_t) \tag{33}$$

$$\text{Policy:} \quad \mathbf{a}_t = \pi(\mathbf{u}_t) \tag{34}$$

$$\text{Closed loop:} \quad \mathbf{u}_{t+1} = f_{\text{NS}}(\mathbf{u}_t, \pi(\mathbf{u}_t)) \tag{35}$$

Without action memory, if π is highly reactive and flow dynamics f_{NS} have resonances at certain frequencies, the closed loop can become unstable:

$$\frac{\partial \pi}{\partial \mathbf{u}} \cdot \frac{\partial f_{\text{NS}}}{\partial \mathbf{a}} > 1 \quad \rightarrow \quad \text{Oscillatory instability} \tag{36}$$

With action memory, the policy becomes:

$$\mathbf{a}_t = \pi(\mathbf{u}_t, \mathbf{a}_{t-1}) \tag{37}$$

This adds **feedback damping**. The policy can learn:

$$\frac{\partial \pi}{\partial \mathbf{a}_{t-1}} < 0 \quad \text{when } |\mathbf{a}_{t-1}| \text{ is large} \tag{38}$$

effectively implementing a learned stabilizer that prevents runaway oscillations.

3.4 Adaptive Filtering vs Fixed Filtering

Remark 2 (Comparison with External Filtering)**.** External low-pass filtering applies a fixed rule:

$$\mathbf{a}_t^{\text{filtered}} = \alpha \mathbf{a}_t^{\text{raw}} + (1 - \alpha)\mathbf{a}_{t-1}^{\text{filtered}} \tag{39}$$

This has two problems:

1. Fixed α may be suboptimal (too smooth or too reactive)
2. Credit assignment is broken: reward depends on $\mathbf{a}_t^{\text{filtered}}$ but policy outputs $\mathbf{a}_t^{\text{raw}}$

With action memory, the policy learns adaptive filtering:

$$\mathbf{a}_t = \pi_\theta(\mathbf{u}_t, \mathbf{w}_t, \mathbf{a}_{t-1}) = f_{\text{base}}(\mathbf{u}_t, \mathbf{w}_t) + \alpha_{\text{learned}}(\mathbf{u}_t, \mathbf{w}_t) \cdot \mathbf{a}_{t-1} \tag{40}$$

where α_{learned} is adaptive and learned through experience.

4 Critic Architecture and Apparent Redundancy

4.1 Critic Input Composition

The critic network receives:

$$\text{Critic input: } [\mathbf{u}_t, \mathbf{w}_t, \mathbf{a}_{t-1}, \mathbf{a}_t] \in \mathbb{R}^{4\times 64\times 64} \tag{41}$$

This appears redundant since:

$$\mathbf{a}_{t-1}^{(t)} = \mathbf{a}_t^{(t-1)} \tag{42}$$

4.2 Why This Redundancy is Necessary

Proposition 3 (Critic Requires Full State and Action)**.** For proper Q-function approximation, the critic must receive both:

1. The complete state: $\mathbf{s}_t = [\mathbf{u}_t, \mathbf{w}_t, \mathbf{a}_{t-1}]$
2. The current action being evaluated: $\mathbf{a}_t$

Proof. The Q-function is defined as:

$$Q : \mathcal{S} \times \mathcal{A} \to \mathbb{R} \tag{43}$$

With augmented state:

$$Q(\mathbf{s}_t, \mathbf{a}_t) = Q([\mathbf{u}_t, \mathbf{w}_t, \mathbf{a}_{t-1}], \mathbf{a}_t) \tag{44}$$

The critic must evaluate *how good* is action $\mathbf{a}_t$ *given* that the previous action was $\mathbf{a}_{t-1}$. Both pieces of information are essential:

- $\mathbf{a}_{t-1}$ (from state): Historical context
- $\mathbf{a}_t$ (from action input): Current decision being evaluated

□

4.3 Information Encoded in Redundancy

The apparent redundancy actually encodes valuable information:

4.3.1 Action Change Magnitude

$$\Delta\mathbf{a}_t = \mathbf{a}_t - \mathbf{a}_{t-1} \tag{45}$$

The critic can learn:

$$Q([\mathbf{u}, \mathbf{w}, \mathbf{a}_{\text{prev}}], \mathbf{a}_{\text{curr}}) = Q_{\text{base}}(\mathbf{u}, \mathbf{w}, \mathbf{a}_{\text{curr}}) - \lambda_Q \left\|\mathbf{a}_{\text{curr}} - \mathbf{a}_{\text{prev}}\right\|^2 \tag{46}$$

4.3.2 Context-Dependent Action Evaluation

The same action $\mathbf{a}_t$ has different value depending on action history:

$$Q([\mathbf{u}, \mathbf{w}, +0.9], +0.8) > Q([\mathbf{u}, \mathbf{w}, -0.7], +0.8) \tag{47}$$

$$\text{(Smooth continuation)} \quad \text{(Large jump)} \tag{48}$$

4.4 Critic Network Architecture

The critic uses circular convolutions to process the full field:

$$Q_\phi(\mathbf{s}_t, \mathbf{a}_t) = \text{MLP}\left(\text{Flatten}\left(\text{Conv}_{\text{circ}}\left(\begin{bmatrix}\mathbf{u}_t \\ \mathbf{w}_t \\ \mathbf{a}_{t-1} \\ \mathbf{a}_t\end{bmatrix}\right)\right)\right) \tag{49}$$

Input channels:

- Without action memory: $[u, w, a_{\text{curr}}]$ — 3 channels
- With action memory: $[u, w, a_{\text{prev}}, a_{\text{curr}}]$ — 4 channels

5 Implementation Details

5.1 Environment Modifications

5.1.1 State Representation

Each agent observes an 8×8 patch:

$$\text{obs}^{(i)} = \begin{cases}[\mathbf{u}^{(i)}, \mathbf{w}^{(i)}] & \text{without action memory (2 channels)} \\ [\mathbf{u}^{(i)}, \mathbf{w}^{(i)}, \alpha \cdot \mathbf{a}_{t-1}^{(i)}] & \text{with action memory (3 channels)}\end{cases} \tag{50}$$

where α is a scaling factor (default: 1.0).

5.1.2 Previous Action Field Tracking

The environment maintains:

$$\mathbf{a}_{\text{field}}^{\text{prev}} \in \mathbb{R}^{64\times 64} \tag{51}$$

Updated after each step:

$$\mathbf{a}_{\text{field}}^{\text{prev}} \leftarrow \mathbf{a}_{\text{field}}^{\text{curr}} \tag{52}$$

The zero-mean constraint is applied in the training script *before* actions reach the environment (see Section 7 for details).

5.1.3 Episode Reset

$$\mathbf{a}_{\text{field}}^{\text{prev}} \leftarrow \mathbf{0}_{64\times 64} \quad \text{at episode start} \tag{53}$$

5.2 Actor Network Architecture

CNN encoder-decoder with variable input channels:

$$\text{Encoder:} \quad \mathbf{h} = f_{\text{enc}}(\text{obs}) \in \mathbb{R}^{C\times 8\times 8} \tag{54}$$

$$\text{Similarity:} \quad \boldsymbol{\phi} = \text{GAP}(g_{\text{sim}}(\mathbf{h})) \in \mathbb{R}^{d_{\text{sim}}} \tag{55}$$

$$\text{Decoder:} \quad \mathbf{a} = \tanh(f_{\text{dec}}(\mathbf{h})) \in [-1, 1]^{8\times 8} \tag{56}$$

Number of input channels:

$$C_{\text{in}} = \begin{cases}2 & \text{without action memory} \\ 3 & \text{with action memory}\end{cases} \tag{57}$$

5.3 Critic Network Architecture

Input preparation:

$$\mathbf{X}_{\text{critic}} = \text{Reconstruct}(\{\text{obs}^{(i)}\}_{i=1}^{64}, \{\mathbf{a}^{(i)}\}_{i=1}^{64}) \in \mathbb{R}^{C_{\text{obs}}+1\times 64\times 64} \tag{58}$$

where $C_{\text{obs}} = 2$ or 3 depending on whether action memory is enabled.
Forward pass:

$$\mathbf{h}_1 = \text{CircularConv}(\mathbf{X}_{\text{critic}}) \tag{59}$$

$$\mathbf{h}_2 = \text{MaxPool}(\text{ReLU}(\text{GroupNorm}(\mathbf{h}_1))) \tag{60}$$

$$\vdots \tag{61}$$

$$Q = \text{Linear}(\text{Flatten}(\mathbf{h}_L)) \in \mathbb{R} \tag{62}$$

5.4 Training Algorithm

Algorithm 1 MADDPG with Action Memory

Initialize actor π_θ, critic Q_ϕ, target networks $\pi_{\theta'}$, $Q_{\phi'}$
Initialize replay buffer $\mathcal{D}$
for episode $= 1, \ldots, N_{\text{episodes}}$ **do**
  Reset environment, set $\mathbf{a}_{-1} = \mathbf{0}$
  for step $= 0, \ldots, T-1$ **do**
    Observe $\mathbf{s}_t = [\mathbf{u}_t, \mathbf{w}_t, \mathbf{a}_{t-1}]$
    Select action $\mathbf{a}_t = \pi_\theta(\mathbf{s}_t) + \mathcal{N}(0, \sigma^2)$
    Execute $\mathbf{a}_t$, observe r_t, $\mathbf{u}_{t+1}$, $\mathbf{w}_{t+1}$
    Store transition $(\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1})$ in $\mathcal{D}$
    if training_step mod train_freq $= 0$ **then**
      for $g = 1, \ldots, G$ **do**
        Sample minibatch $\{(\mathbf{s}_j, \mathbf{a}_j, r_j, \mathbf{s}_{j+1})\}$ from $\mathcal{D}$
        Critic update:
          Compute target: $y_j = r_j + \gamma Q_{\phi'}(\mathbf{s}_{j+1}, \pi_{\theta'}(\mathbf{s}_{j+1}))$
          Update: $\phi \leftarrow \phi - \eta_Q \nabla_\phi \frac{1}{B} \sum_j (Q_\phi(\mathbf{s}_j, \mathbf{a}_j) - y_j)^2$
        Actor update:
          $\mathcal{L}_{\text{actor}} = -\frac{1}{B} \sum_j Q_\phi(\mathbf{s}_j, \pi_\theta(\mathbf{s}_j)) + \lambda_s \mathcal{L}_{\text{spatial}} + \lambda_c \mathcal{L}_{\text{consistency}}$
          Update: $\theta \leftarrow \theta - \eta_\pi \nabla_\theta \mathcal{L}_{\text{actor}}$
        Target update:
          $\theta' \leftarrow \tau\theta + (1-\tau)\theta'$
          $\phi' \leftarrow \tau\phi + (1-\tau)\phi'$
      end for
    end if
  end for
end for

6 Backwards Compatibility

6.1 Configuration-Based Selection

Action memory is enabled via configuration:

```
observation:
  include_prev_action: true   # Enable (default: false)
  prev_action_scale: 1.0      # Scaling factor
```

6.2 Automatic Channel Detection

The system automatically adapts:

$$C_{\text{in}}^{\text{actor}} = \text{obs\_shape}[0] \in \{2, 3\} \tag{63}$$

$$C_{\text{in}}^{\text{critic}} = C_{\text{in}}^{\text{actor}} + 1 \in \{3, 4\} \tag{64}$$

6.3 Network Initialization

Configuration	Actor Input	Critic Input
Without action memory	$[u, w]$ (2 ch)	$[u, w, a_{\text{curr}}]$ (3 ch)
With action memory	$[u, w, a_{\text{prev}}]$ (3 ch)	$[u, w, a_{\text{prev}}, a_{\text{curr}}]$ (4 ch)

Table 1: Channel configuration for different modes

7 Expected Results

7.1 Smoothness Metrics

Metric	Without Memory	With Memory (Expected)
Temporal gradient RMS	≈ 0.4	0.1–0.2
High-freq FFT content	High	Reduced
Action change rate	$> 50\%$ steps	$< 20\%$ steps

Table 2: Expected smoothness improvements

7.2 Performance Metrics

Important: Drag reduction may slightly *decrease* compared to bang-bang:

$$\mathbb{E}[R_{\text{smooth}}] \approx \mathbb{E}[R_{\text{bang-bang}}] \quad \text{or slightly lower} \tag{65}$$

This is an **acceptable trade-off** because:

1. Bang-bang is not physically realizable (actuator limitations)
2. Smooth control is more robust to model mismatch
3. Avoids exploiting simulation-specific resonances
4. Breaks unstable feedback loop with wave dynamics

The goal is **not** to maximize drag reduction at all costs, but to find *smooth, robust, physically realizable* control that still achieves good performance.

Total objective balances drag reduction with smoothness:

$$J = \mathbb{E}[R_{\text{drag}}] - \lambda_s \mathbb{E}[\mathcal{L}_{\text{spatial}}] - \lambda_c \mathbb{E}[\mathcal{L}_{\text{consistency}}] \tag{66}$$

Expected outcomes:

- Drag reduction: 90%–100% of bang-bang performance (acceptable)
- Temporal smoothness: 4× improvement (crucial)
- Physical realizability: Yes (vs no for bang-bang)
- Robustness: Improved (no feedback instability)

7.3 Learning Dynamics

Expected training progression:

1. **Early training** (episodes 0–50): Policy ignores $\mathbf{a}_{t-1}$, still jumpy
2. **Mid training** (episodes 50–150): Policy starts using $\mathbf{a}_{t-1}$ context
3. **Late training** (episodes 150+): Smooth control emerges, temporal consistency learned

8 Zero-Mean Constraint: Critical Implementation Details

8.1 The Problem with Post-Processing Correction

Critical Discovery: The standard approach of applying zero-mean correction as post-processing can *itself cause* bang-bang behavior!

8.1.1 How External Correction Creates Oscillations

Standard implementation (PROBLEMATIC):

Algorithm 2 Post-Processing Zero-Mean (Problematic)

1: Agents output raw actions: $\mathbf{a}_i^{\text{raw}}$ for $i = 1, \ldots, 64$
2: Collect into field: $\mathbf{A}^{\text{raw}} \in \mathbb{R}^{64\times 64}$
3: Compute mean: $\mu = \frac{1}{N}\sum_{i,j} A_{ij}^{\text{raw}}$
4: Apply correction: $\mathbf{A}^{\text{exec}} = \mathbf{A}^{\text{raw}} - \mu$
5: Execute $\mathbf{A}^{\text{exec}}$, store in $\mathbf{a}_{\text{prev}}$ for next step

The oscillation mechanism:

$$t = 0: \quad \mu_0 = +0.3 \quad \rightarrow \quad \mathbf{A}_0^{\text{exec}} = \mathbf{A}_0^{\text{raw}} - 0.3 \quad \text{(shift DOWN)} \tag{67}$$

$$t = 1: \quad \mu_1 = -0.2 \quad \rightarrow \quad \mathbf{A}_1^{\text{exec}} = \mathbf{A}_1^{\text{raw}} + 0.2 \quad \text{(shift UP)} \tag{68}$$

$$t = 2: \quad \mu_2 = +0.4 \quad \rightarrow \quad \mathbf{A}_2^{\text{exec}} = \mathbf{A}_2^{\text{raw}} - 0.4 \quad \text{(shift DOWN)} \tag{69}$$

Even if $\mathbf{A}_t^{\text{raw}}$ is smooth, **varying μ_t creates oscillating $\mathbf{A}_t^{\text{exec}}$!**

8.2 Gradient Mismatch Problem

With post-processing correction:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial \mathbf{A}^{\text{exec}}} \cdot \frac{\partial \mathbf{A}^{\text{exec}}}{\partial \theta} \tag{70}$$

But gradient computation assumes:

$$\mathbf{A}^{\text{exec}} = \pi_\theta(\mathbf{s}) \quad \text{(direct control)} \tag{71}$$

Reality with correction:

$$\mathbf{A}^{\text{exec}} = \pi_\theta(\mathbf{s}) - \frac{1}{N}\sum_{i=1}^{N} \pi_\theta(\mathbf{s}_i) \quad \text{(indirect, coupled)} \tag{72}$$

The Jacobian is:

$$\frac{\partial A_i^{\text{exec}}}{\partial A_j^{\text{raw}}} = \begin{cases} 1 - \frac{1}{N} & i = j \\ -\frac{1}{N} & i \neq j \end{cases} \tag{73}$$

Gradients don't see this coupling! Policy learns as if it directly controls executed actions.

8.3 Solution: Differentiable Zero-Mean Correction

Make the correction part of the computational graph:

Algorithm 3 Differentiable Zero-Mean (Correct)

1: Forward pass all agents: $\mathbf{A}^{\text{raw}} = \pi_\theta(\mathbf{S})$ (from tanh $\rightarrow [-1, 1]$)
2: Compute mean (differentiable): $\mu = \text{mean}(\mathbf{A}^{\text{raw}})$
3: Apply correction (differentiable): $\mathbf{A} = \mathbf{A}^{\text{raw}} - \mu$
4: Compute loss: $\mathcal{L} = -Q(\mathbf{S}, \mathbf{A}) + \lambda_s \mathcal{L}_{\text{spatial}} + \cdots$
5: Backpropagate: $\nabla_\theta \mathcal{L}$ includes effect of zero-mean constraint
6: Update: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$

Key benefits:

1. Gradients properly account for zero-mean constraint
2. Policy learns to output actions with near-zero mean *before* correction
3. Correction magnitude $|\mu|$ becomes smaller during training
4. Bang-bang oscillations from varying corrections are prevented

8.3.1 Interaction with Exploration Noise and Clipping

During training, exploration noise is added:

$$\mathbf{A}^{\text{noisy}} = \mathbf{A}^{\text{raw}} + \mathcal{N}(0, \sigma^2 I) \tag{74}$$

where $\mathbf{A}^{\text{raw}} \in [-1, 1]$ (from tanh activation). Adding noise can push actions outside $[-1, 1]$, requiring clipping:

$$\mathbf{A}^{\text{clipped}} = \text{clip}(\mathbf{A}^{\text{noisy}}, -1, 1) \tag{75}$$

Critical issue: Clipping breaks the zero-mean property! If $\mathbf{A}^{\text{noisy}}$ is zero-mean, $\mathbf{A}^{\text{clipped}}$ may not be.

Solution: Apply zero-mean correction *after* clipping:

$$\mathbf{A}^{\text{final}} = \mathbf{A}^{\text{clipped}} - \text{mean}(\mathbf{A}^{\text{clipped}}) \tag{76}$$

Important: When noise is annealed to zero ($\sigma = 0$ in late training), clipping is unnecessary:

- Actor outputs via tanh are already in $[-1, 1]$
- No noise added $\rightarrow$ actions remain in $[-1, 1]$
- Clipping step can be skipped
- Zero-mean correction still applied as safety net (ensures exact zero-mean)

Full procedure:

8.4 Per-Tile vs Global Zero-Mean Constraint

8.4.1 Per-Tile Constraint is Physically Invalid

Previously used (WRONG):

$$\mathcal{L}_{\text{zero}} = \frac{1}{N_{\text{agents}}} \sum_{i=1}^{N_{\text{agents}}} (\text{mean}(\mathbf{a}_i))^2 \tag{77}$$

This penalizes non-zero mean per 8×8 tile.

Why this is invalid: Different flow regions require different average actuation:

Algorithm 4 Differentiable Zero-Mean with Noise Handling

Forward pass: $\mathbf{A}^{\text{raw}} = \pi_\theta(\mathbf{S})$ $(\in [-1, 1]$ from tanh)
if training mode with noise **then**
  Generate zero-mean noise: $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \sigma^2 I)$, $\boldsymbol{\epsilon} \leftarrow \boldsymbol{\epsilon} - \text{mean}(\boldsymbol{\epsilon})$
  Add noise: $\mathbf{A}^{\text{noisy}} = \mathbf{A}^{\text{raw}} + \boldsymbol{\epsilon}$ (may exceed $[-1, 1]$)
  Clip: $\mathbf{A}^{\text{clipped}} = \text{clip}(\mathbf{A}^{\text{noisy}}, -1, 1)$ (breaks zero-mean!)
  Guarantee zero-mean: $\mathbf{A} = \mathbf{A}^{\text{clipped}} - \text{mean}(\mathbf{A}^{\text{clipped}})$
else
  No clipping needed (already in $[-1, 1]$)
  Guarantee zero-mean: $\mathbf{A} = \mathbf{A}^{\text{raw}} - \text{mean}(\mathbf{A}^{\text{raw}})$
end if
Execute $\mathbf{A}$ (guaranteed zero-mean)

- Upstream regions may need net blowing: $\text{mean}(\mathbf{a}_{\text{upstream}}) > 0$
- Downstream regions may need net suction: $\text{mean}(\mathbf{a}_{\text{downstream}}) < 0$
- Separated regions require strong local forcing

Forcing each tile to zero-mean is an **unphysical artificial constraint**!

8.4.2 Global Zero-Mean is Required

Correct approach (mass conservation):

$$\mathcal{L}_{\text{global\_mean}} = \left(\frac{1}{64 \times 64} \sum_{i,j} A_{ij} \right)^2 \tag{78}$$

This enforces:

$$\text{mean}(\mathbf{A}_{\text{field}}) = 0 \quad \text{(incompressibility)} \tag{79}$$

while allowing:

$$\text{mean}(\mathbf{a}_i) \neq 0 \quad \text{for individual tiles} \tag{80}$$

8.5 Implementation

8.5.1 In Actor Network

During training with exploration noise:

```
# Forward pass for all agents
actions_raw = actor(obs_batch)  # [batch, 64, 8, 8], in [-1, 1] from tanh

# Add zero-mean exploration noise
if noise_scale > 0:
    noise = np.random.normal(0, noise_scale, size=actions_raw.shape)
    noise = noise - noise.mean()  # Ensure noise is zero-mean
    actions = actions_raw + noise  # May exceed [-1, 1]
    actions = np.clip(actions, -1, 1)  # Clipping needed
else:
    actions = actions_raw  # No clipping needed (already in [-1, 1])

# CRITICAL: Guarantee exact zero-mean after clipping
```

```
# (clipping can break zero-mean property)
actions = actions - actions.mean()

# Actions are now:
# 1. Exactly zero-mean (guaranteed)
# 2. In [-1, 1] range
# 3. Gradients flow through correction (differentiable)
```

During inference without noise (late training or evaluation):

```
# Forward pass
actions_raw = actor(obs)  # In [-1, 1] from tanh

# No noise, no clipping needed
# Only zero-mean correction for exact guarantee
global_mean = actions_raw.mean()
actions = actions_raw - global_mean

return actions  # Guaranteed zero-mean, in [-1, 1]
```

Key points:

- Clipping only required when adding noise ($\sigma > 0$)
- Without noise, tanh output is naturally in $[-1, 1]$
- Final zero-mean correction always applied (cheap safety net)
- Ensures exact $\text{mean}(\mathbf{A}) = 0$ for solver stability

8.5.2 In Loss Function

Old approach (WRONG):

$$\mathcal{L}_{\text{actor}} = -Q + \lambda_{\text{spatial}}\mathcal{L}_{\text{spatial}} + \lambda_{\text{zero}} \underbrace{\sum_i (\text{mean}(\mathbf{a}_i))^2}_{\text{per-tile, invalid}} \tag{81}$$

New approach (CORRECT):

$$\mathcal{L}_{\text{actor}} = -Q + \lambda_{\text{spatial}}\mathcal{L}_{\text{spatial}} + \lambda_{\text{global}} \underbrace{(\text{mean}(\mathbf{A}_{\text{field}}))^2}_{\text{global only, valid}} \tag{82}$$

8.6 Expected Impact

With these fixes:

1. **Reduced bang-bang from corrections**: Policy learns to output near-zero mean $\rightarrow$ small $|\mu| \rightarrow$ stable corrections
2. **Proper credit assignment**: Gradients include zero-mean constraint effect
3. **Physical validity**: Tiles can have non-zero mean as required by flow physics
4. **Training stability**: Network aware of constraint during learning
5. **Exact zero-mean guarantee**: Final correction after clipping ensures $\text{mean}(\mathbf{A}) = 0$ exactly for solver stability
6. **Efficient inference**: When noise is annealed ($\sigma \rightarrow 0$), clipping is unnecessary since tanh output is naturally in $[-1, 1]$

Approach	Gradients See Constraint	Physically Valid
Post-processing + per-tile loss	No	No
Differentiable + per-tile loss	Yes	No
Post-processing + global loss	No	Yes
Differentiable + global loss	Yes	Yes

Table 3: Comparison of zero-mean constraint approaches

9 Conclusion

9.1 Summary

We have shown that augmenting the state with previous action:

$$\mathbf{s}_t = [\mathbf{u}_t, \mathbf{w}_t] \quad \rightarrow \quad \mathbf{s}_t = [\mathbf{u}_t, \mathbf{w}_t, \mathbf{a}_{t-1}] \tag{83}$$

provides the policy with the necessary information to learn smooth temporal control while:

1. Preserving the Markov property
2. Enabling adaptive (learned) temporal filtering
3. Maintaining clean credit assignment
4. Requiring minimal code changes
5. Ensuring full backwards compatibility

9.2 Key Advantages Over Alternatives

Approach	Markovian	Credit Assignment	Adaptive
Temporal loss on replay	No	Confused	Partial
External filtering	Yes	Broken	No
Prev action in obs	Yes	Clean	Yes

Table 4: Comparison of approaches to temporal smoothness

9.3 Future Work

Potential extensions:

1. Multiple-step action history: $\mathbf{s}_t = [\mathbf{u}_t, \mathbf{w}_t, \mathbf{a}_{t-1}, \mathbf{a}_{t-2}]$
2. Learned scaling: $\alpha(\mathbf{u}_t, \mathbf{w}_t)$ instead of fixed α
3. Action delta encoding: $[\mathbf{u}_t, \mathbf{w}_t, \Delta\mathbf{a}_{t-1}]$ instead of absolute $\mathbf{a}_{t-1}$